

TokenPack: 0/1 Knapsack Algorithm Analysis and LLM Context-Packing Experiments

Metehan Kizilcik

Department of Software Engineering

Manisa Celal Bayar University (CBU)

Email: 232803055@ogr.cbu.edu.tr, metekizilcik@gmail.com

Code: <https://github.com/mo-tunn/TokenPack>

Abstract—This paper studies the 0/1 Knapsack Problem as a canonical NP-Hard optimization problem and evaluates exact and heuristic algorithms under increasing instance sizes. The main algorithmic comparison uses repeated synthetic knapsack instances with $N \in \{100, 500, 1000, 5000, 10000\}$ and measures runtime, timeout behavior, objective value, and accuracy gap from the dynamic-programming optimum when that optimum is computationally feasible. Exact dynamic programming gives the optimum for manageable instances but becomes impractical as the n^C state space grows, while density-aware greedy and simulated annealing remain efficient with very small gaps on the tested solvable sizes. The second experimental part applies the same budgeted-selection view to TokenPack, an LLM context-packing pipeline where document chunks are items, token counts are weights, and query-conditioned salience is value. The LLM experiments are presented as an application case study, not as a statistically powered retrieval benchmark: they show that knapsack is a useful abstraction for budgeted context selection, but they do not support a claim that knapsack-based selection universally dominates simpler budget-aware retrieval. The strongest practical result is selection-first: on an exploratory 83-case LongBench v2 pilot using Qwen/Qwen2.5-14B-Instruct, TokenPack hybrid-greedy answers 37/83 cases correctly at about 50% source-token saving, compared with 32/83 for full-context prompting and 34/83 for production-RAG-50. This is a descriptive 15.6% relative accuracy gain over full context and 8.8% over production-RAG-50, while the TokenPack-then-LongLLMLingua cascade preserves the same 37/83 correctness at 74.6% context saving.

Index Terms—Knapsack, 0/1 Knapsack, NP-Hard, Greedy Algorithm, Dynamic Programming, Simulated Annealing, LLM, RAG, Algorithm Analysis

I. INTRODUCTION

NP-Hard problems such as the Traveling Salesman Problem, Warehouse Location Problem, and Knapsack Problem are commonly used in algorithm analysis to compare exact and heuristic algorithms. This study focuses on the 0/1 Knapsack Problem. The primary objective is to analyze how dynamic programming, greedy heuristics, randomized feasible selection, and simulated annealing behave as the number of items and the capacity-driven state space grow.

To make the optimization problem concrete beyond synthetic item lists, the paper also uses a modern application scenario: context window selection for large language models (LLMs). When a long document is divided into candidate chunks, only a subset can be placed into the limited token budget of an LLM. This selection task naturally follows a knapsack structure.

TokenPack models the context window as a capacity, text chunks as items, token counts as weights, and query relevance scores as values.

The central research question is therefore two-level. First, for the classical 0/1 Knapsack Problem, when does exact DP become too expensive and how close do heuristics stay to the optimum? Second, in the LLM application case study, does the knapsack abstraction help explain the trade-off between relevance value and token cost? This framing keeps the algorithm analysis separate from the application claim: the paper evaluates knapsack solvers rigorously, then tests whether the same abstraction is useful for retrieval-time context packing.

The contribution of this work is fourfold. First, it implements and compares exact DP, greedy, random feasible, and simulated annealing algorithms on repeated 0/1 knapsack instances. Second, it reports runtime, timeout rate, and accuracy gap across increasing item counts, including the point where DP becomes impractical. Third, it presents a direct mapping from LLM context selection to knapsack as an application case study. Fourth, it evaluates chunking, scoring, selection, and compression choices on evidence-oriented retrieval and long-context generation benchmarks, showing where the knapsack abstraction is helpful and where simpler budget-aware heuristics remain stronger.

II. LITERATURE REVIEW

The Knapsack Problem is one of the fundamental NP-Hard problems in combinatorial optimization [1], [2]. Many extensions and solution strategies have been studied, including multidimensional, multiobjective, and multiple-choice variants [3]–[5]. Metaheuristic optimization methods such as NSGA-II are also frequently used for combinatorial optimization when exact methods are too expensive [6].

Retrieval-Augmented Generation (RAG) provides external knowledge to LLMs by retrieving relevant document chunks [7]. However, simple top-k retrieval does not directly optimize token budget usage. Long-context methods such as MemGPT, Parallel Context Windows, LongRoPE, PoSE, and positional interpolation show that context length is a major practical limitation in LLM systems [8]–[12]. Recent evaluations also compare long-context models and RAG, and show that effective context length and computational cost still matter even when nominal context windows grow [13]–[16].

Chunking and embedding quality are also central to retrieval. Late Chunking, S2 Chunking, Landmark Embedding, and document chunking studies demonstrate that segmentation and representation choices affect downstream retrieval quality [17]–[21]. Context pruning and query-aware sparsity methods such as AttentionRAG, Quest, SnapKV, WindowKV, and BudgetMem further support the idea that not every token should be placed into the model context [22]–[26]. Prompt-compression methods such as LLMingua, LongLLMLingua, and LLMingua-2 attack the same budget problem from a different angle: instead of selecting chunks before prompting, they compress the prompt tokens after context construction [27]–[29]. BudgetMem is especially relevant because it keeps selection at the chunk level, uses BM25 retrieval after ingestion-time filtering, and scores chunks with interpretable signals such as TF-IDF, position bias, entity density, numerical density, and discourse markers. These neighboring lines of work motivate a broader TokenPack question: the optimizer should be tested not only with different solvers, but also against compression-first alternatives that operate at a different stage of the pipeline.

Knapsack-style formulations have also appeared in summarization and recent LLM optimization work. Tree knapsack summarization and budget-aware summarization explicitly connect content selection with budget constraints [30], [31]. Recent LLM-oriented work uses knapsack-like ideas for NP-Hard problem evaluation, exploration budget allocation, and speculative decoding [32]–[34]. TokenPack follows this direction but focuses specifically on retrieval-time context packing.

III. PROBLEM FORMULATION

The core optimization problem is the standard 0/1 Knapsack Problem. There are n items, each item i has a nonnegative value v_i and a positive weight w_i , and the knapsack has capacity C . The goal is to select a subset with maximum total value while keeping total weight within capacity.

In the LLM context-packing case study, a query q and a document define the item set. The document is split into n chunks. Each chunk i has value v_i and weight w_i . In the default TokenPack scoring setting, w_i is the chunk token count and v_i is the production *evidence-hybrid* score:

$$v_i = 0.35c_i + 0.25b_i + 0.20q_i + 0.15s_i + 0.05h_i \quad (1)$$

where c_i is normalized embedding cosine similarity, b_i is normalized BM25 lexical relevance, q_i is explicit query-term coverage, s_i is a structural prior from source metadata, and h_i is adjacent-chunk coherence. These coefficients are hand-set rather than learned: they intentionally combine semantic similarity, lexical matching, evidence coverage, and structural cues. Their relative weights were chosen by engineering judgment and ablation feedback, not by a systematic calibration procedure. They should therefore be read as a transparent evidence-selection heuristic, not as a tuned optimum. The effective LLM context budget is denoted by C .

Historical ablations also test cosine-only, hybrid, and knapsack-aware scoring profiles. These profiles are kept in an experimental module to test score alignment, not to claim that their coefficients are theoretically optimal; no hyperparameter sensitivity sweep is claimed in this version. The revised artifact uses `sentence-transformers/all-MiniLM-L6-v2` as the default embedding model [35], [36]; deterministic hash embeddings are no longer used for paper-grade runs.

The optimization problem is:

$$\max \sum_{i=1}^n v_i x_i \quad (2)$$

$$\sum_{i=1}^n w_i x_i \leq C, \quad x_i \in \{0, 1\} \quad (3)$$

Here, $x_i = 1$ means that item or chunk i is selected. In the synthetic experiments, v_i and w_i are generated numerical item attributes. In the LLM case study, the objective maximizes total estimated semantic utility while preserving the hard token budget. This formulation is more informative than fixed top-k retrieval when chunk lengths are unequal, because top-k ignores item weight.

IV. METHOD

The TokenPack pipeline has four main stages:

- 1) The long document is split into chunks using the default structure-aware chunker, which preserves structural boundaries and can split within a section when adjacent block embeddings indicate semantic drift.
- 2) Each chunk and query is embedded into a vector space.
- 3) Chunk value is computed using either cosine similarity or the hybrid salience score, and chunk weight is computed using token count.
- 4) Selection is performed with top-k, budget-top-k, MMR, greedy heuristics, or 0/1 knapsack dynamic programming.

A. Semantic Chunking

Chunking is the stage that converts an unstructured long document into knapsack items. This is important because the optimization algorithm can only choose among the chunks that the preprocessing stage creates. If chunk boundaries are poor, even an optimal knapsack solver may select incomplete or semantically broken evidence.

TokenPack uses **structure-aware semantic chunking** as the default. The document is first split into structural blocks such as paragraphs, PDF text blocks, sections, and code symbols. Consecutive document blocks are grouped until the chunk reaches a target token range, but the chunker preserves file boundaries, content-type changes, section hints, and function/class boundaries when source code metadata is available. Each chunk keeps metadata such as source path, page number when available, block order, character offsets, and line spans for code. This metadata is important because selected chunks are later restored to their original document order before being sent to the LLM.

Within a structurally compatible run of document blocks, the default chunker also uses a semantic drift check. For two adjacent blocks b_i and b_{i+1} , TokenPack computes:

$$s_i = \cos(E(b_i), E(b_{i+1})) \quad (4)$$

where $E(\cdot)$ is the embedding function. If s_i falls below a threshold and the current chunk has reached a minimum useful size, the algorithm treats this point as a topic shift and starts a new chunk. Structural boundaries take precedence over semantic boundaries: semantic drift is used to refine otherwise compatible document blocks, not to merge across sections or split code symbols arbitrarily. A maximum token limit is still enforced, so chunking cannot create chunks that violate the later token-budget assumptions.

Paragraph-only and standalone semantic-threshold chunkers remain as ablation baselines. The purpose of chunking is not to solve the optimization problem by itself; it improves the quality of the item set given to knapsack. The output item must contain three values needed by the optimizer: text content, token weight, and source-order metadata.

B. Compared Algorithms

Dynamic Programming (DP). DP computes the optimal subset under the token capacity. The implementation uses a one-dimensional value table for the objective and a reconstruction table for selected weight. Its time complexity is $O(nC)$.

Greedy by Value. This heuristic sorts chunks by value and inserts each chunk if it fits. It is fast, but it may choose large chunks that block better combinations.

Greedy by Value Density. This heuristic sorts chunks by v_i/w_i . It is usually stronger than value-only greedy because it accounts for token efficiency, but it is still not guaranteed to find the optimal 0/1 solution.

Greedy by Lightest Weight. This baseline chooses short chunks first. It tests whether token minimization alone is enough. In practice, it ignores semantic value and therefore may select weak evidence.

Random Feasible. This baseline shuffles chunks and inserts feasible items. It gives a lower-bound style comparison for non-optimized selection.

Simulated Annealing (SA). SA starts from a density-greedy feasible solution and then explores neighboring subsets by adding or removing one item at a time. It uses temperature $\max(1, n/2)$, $\min(25000, \max(2000, 12n))$ iterations, multiplicative cooling by 0.9995, and a 1.02 deterministic reheating every 250 steps. This provides a metaheuristic baseline that is more exploratory than simple greedy selection.

Document-prefix and Full-document Baselines. These two baselines represent the system without TokenPack-style optimization. Document-prefix inserts chunks in the original PDF order until the budget is full. Full-document tries to insert the entire source document. These baselines are useful because they show what happens when semantic value and token cost are not optimized jointly.

Production-RAG, Top-k, Budget-top-k, and MMR. The production-RAG baseline represents a common vector-retrieval

Table I
PAIRED STATISTICAL COMPARISON OF ALGORITHMS FOR $N = 1000$ OVER 100 RUNS

Algorithm	Mean Value	Mean Time (ms)	Gap (%)	95% CI Gap	Speedup
DP exact	111281.2	56.644	0.00	0.00	–
Greedy density	111252.4	0.209	0.03	0.01	271.3x
Simulated annealing	111252.4	3.784	0.03	0.01	15.0x
Greedy lightest	95924.7	0.150	13.84	0.38	376.4x
Greedy value	40785.1	0.147	63.30	0.65	386.4x
Random feasible	20897.2	0.173	81.20	0.43	326.8x

RAG system: it ranks chunks only by dense embedding cosine similarity to the query, then inserts the highest-similarity chunks until the token budget is full. It does not rewrite, summarize, or compress chunk text; it reduces context length only by selecting fewer chunks. A label such as `production-rag-50` means that this similarity-ranked retrieval baseline receives a source-relative token budget of about 50% of the original source context. Top-k selects a fixed number of chunks without checking budget. Budget-top-k uses TokenPack’s evidence-hybrid value score rather than raw dense similarity, then adds the same budget-feasibility check. MMR additionally penalizes redundancy between selected chunks. The `knapsack-redundancy` strategy keeps MMR as a separate algorithm, but applies a redundancy penalty to chunk values before the knapsack solver runs. This is a practical linear approximation rather than a full quadratic knapsack formulation.

These algorithms were not only described theoretically; they were executed on the same generated instances so that their differences can be measured statistically. Table I reports a paired comparison for $N = 1000$ over 100 runs. The word “paired” means that every algorithm receives the same item values, weights, and capacity for each repetition. Therefore, the measured gap is caused by the algorithmic strategy rather than by different random inputs. In the tables, shaded rows mark the TokenPack-relevant exact knapsack baseline and its practical density-based heuristic.

The comparison shows a clear trade-off. DP is the exact baseline and has zero gap, but it is much slower. Greedy density is approximately 271 times faster than DP in this setting while keeping the mean gap at only 0.03%. Simulated annealing reaches a similar gap in this experiment, but it is slower than density-greedy because it performs additional neighborhood exploration. Greedy value is also fast, but its mean gap is 63.30%, which shows that choosing high-value items without considering weight is unreliable. The lightest-first baseline is faster than DP but loses 13.84% value on average, proving that minimizing tokens alone is also not enough.

C. Why Knapsack is Appropriate

Knapsack is appropriate for this study because it is a canonical NP-Hard problem with a clear exact dynamic-programming baseline and several practical heuristic alternatives. This makes it suitable for comparing runtime, scalability, and approximation quality under the same generated instances.

The mapping from context selection to knapsack is also direct. The context window is the capacity, chunks are items,

token counts are weights, and query relevance scores are values. This model is more faithful than fixed top-k retrieval because it optimizes estimated information value under an explicit constraint. However, the LLM setting also shows an important limitation: optimizing the additive knapsack objective is only useful if the value function is well aligned with downstream answer quality.

V. COMPLEXITY ANALYSIS

For n candidate chunks and effective token budget C , the DP algorithm has $O(nC)$ time complexity. This is pseudo-polynomial: it is efficient for moderate token budgets but can become expensive when either the number of chunks or the capacity is large. Greedy algorithms require sorting, so their time complexity is $O(n \log n)$ and memory usage is low.

This theoretical expectation leads to a practical hybrid design. Embedding similarity first prunes the candidate pool, then an exact solver or a strong heuristic is applied only to the strongest candidates. This keeps the optimization stage feasible while preserving the main advantage of budget-aware selection.

VI. EXPERIMENTAL DESIGN

Eight main experiment groups are used, and the LongBench generation group includes one larger-context diagnostic extension. The first repeated synthetic benchmark is the main classical algorithm-analysis experiment. The remaining experiments use LLM context selection as an application case study for the same budgeted-selection abstraction.

Repeated synthetic knapsack benchmark. Random instances are generated for $N \in \{100, 500, 1000, 5000, 10000\}$. Each setting is repeated 100 times with different random seeds. DP is treated as the exact baseline when $nC \leq 5,000,000$ states. For larger instances, DP is marked as timeout to represent the practical point where the exact method becomes too expensive. The reported metrics are mean objective value, mean runtime, standard deviation of runtime, mean accuracy gap from DP optimum, maximum gap, and timeout rate.

Chunking ablation benchmark. A sample of PDFs from the `resources` directory is converted into sentence-level source blocks and indexed two ways: standalone semantic-threshold chunking and structure-aware semantic chunking. The evidence targets are defined before chunking as source text spans represented by content terms, so the evaluation does not favor either chunker by using chunk IDs from one index. The solver is fixed to knapsack, scoring is fixed to the evidence-hybrid profile, and budgets are set as 1%, 3%, and 5% of source tokens. This isolates the effect of chunk boundaries from the effect of the selection algorithm and the value function.

Value-function ablation benchmark. The same PDF sample and evidence-term targets are also used to compare several scoring profiles, from cosine-only similarity to richer hybrid and evidence-aware variants. In this experiment, chunking is fixed to structure-aware and selection is fixed to knapsack. Therefore, the only changed variable is the value function used by the optimizer.

QASPER selector ablation. To avoid relying only on the small manually reviewed PDF benchmark, TokenPack is also evaluated on QASPER validation examples with evidence annotations [37]. The revised selector harness compares production-RAG, budget-top-k, greedy-density, knapsack, and knapsack-redundancy. Chunking is fixed to structure-aware and scoring is fixed to the evidence-hybrid value function for TokenPack selectors, while production-RAG uses raw dense similarity ranking. Budgets are defined as 20%, 30%, 40%, and 50% of each paper’s source tokens.

QASPER cost-quality proxy. The main practical goal is to reduce input-token cost while preserving answer quality. Because no local or cloud LLM is assumed to be available for every run, this experiment measures answerability proxies on QASPER. For each selector and budget, it records selected-token cost, retained gold-answer token coverage, retained evidence coverage, and the rate at which both answer and evidence are preserved. The answer-token-overlap F1 metric is computed between the selected context text and the gold answer string after lowercasing and token matching. It is therefore a retention proxy rather than a model-output quality score, and its absolute values are expected to be low because the retained context is much longer than the short gold answer and some QASPER answers are mildly abstractive. Scores are reported relative to a 100% high-budget RAG setting using the same chunker, scoring function, and selector.

QASPER compression comparison. Selection and compression solve related but distinct problems, so they are compared directly on the same QASPER questions. The 200-question subset is not sampled randomly: the script streams QASPER validation papers in dataset order and stops after the first 200 questions from papers with parseable blocks and questions. Three pipelines are evaluated: Only TokenPack, Only LLMingua-2, and TokenPack followed by LLMingua-2. The metrics are final token saving, evidence recall, complete-evidence rate, answer-token-overlap F1, latency, and per-question win rates.

LongBench v2 generation pilot. To test whether the evidence-retention signal transfers to downstream long-context answering, a separate Modal-based experiment evaluates the eligible LongBench v2 multiple-choice cases [38]. The generator is Qwen2.5-14B-Instruct [39]. Source contexts are filtered to the 8k–24k token range so that full-context prompting remains feasible within the configured 32k model length. The released split used for this pilot contains 503 examples; the script streams examples in dataset order and yields 83 eligible cases after skipping 420 examples above 24k source tokens and none below 8k. Five methods are compared: full context, production-RAG at a 50% source-token budget, TokenPack hybrid-greedy at the same 50% budget with score-then-source context ordering, LongLLMingua at rate 0.50, and TokenPack followed by LongLLMingua at rate 0.50. Here, the suffix “50” denotes a selection budget equal to approximately half of each example’s source tokens, not a model-context-window percentage and not a token-level compression rate. The LongLLMingua baseline is run in context-level filtering mode,

with token-level filtering disabled, so it should be interpreted as a practical LongLLMLingua-style context-pruning baseline rather than a full token-level compression study. Accuracy is exact multiple-choice correctness after parsing the model’s A/B/C/D answer.

A larger-context diagnostic reuses the same Modal harness on 32k–58k and 64k–112k source-token windows. It compares full context, production-RAG-50, raw-similarity dynamic programming, evidence-hybrid greedy fill, and evidence-hybrid dynamic programming. These runs contain 30 and 8 cases, respectively, so they are reported as scaling diagnostics rather than as primary accuracy claims.

Illustrative input-cost estimate. A final accounting table scales the LongBench v2 context-reduction ratios to a 1M baseline-token scenario using public API pricing assumptions [40]. This estimate is included only to show the economic implication of token reduction; it is not used as evidence for answer quality.

Reproducibility. The implementation is included in the accompanying artifact as the `tokenpack` Python package, with a source manifest listing modules, tests, Modal commands, and QASPER reproduction scripts. Until a public repository URL is attached, the package claim should be interpreted as artifact-based rather than independently archival.

VII. EXPERIMENTAL RESULTS

Figure 1 summarizes the application-side operating points at a glance: LongBench answer quality versus context size, QASPER evidence preservation, and hot-model latency. Figure 2 visualizes runtime behavior for the classical algorithm benchmark. The y-axis is logarithmic because DP grows much faster than greedy methods, matching the theoretical difference between $O(nC)$ dynamic programming and $O(n \log n)$ greedy sorting.

Figure 3 shows the approximation gap for $N = 1000$. Greedy density is visually almost indistinguishable from DP in terms of objective value, while greedy value and random feasible selection lose a large portion of the optimum. This is the most important quality comparison: not every fast heuristic is equally reliable.

Table II reports the same repeated synthetic results numerically. For $N = 100$ and $N = 1000$, DP gives the optimum but is slower than greedy methods. At $N = 1000$, DP takes 56.644 ms on average, while greedy density takes only 0.209 ms. However, greedy density remains very close to optimum, with only 0.03% mean gap. Greedy value is much weaker: its mean gap reaches 63.30% at $N = 1000$.

Figure 4 shows why DP cannot be used blindly on very large candidate pools. The x-axis is the DP state space nC . The red timeout points appear after the configured state threshold. This directly illustrates the pseudo-polynomial limitation of the exact method.

Table III gives the exact state counts behind Figure 4. DP works for 50,000, 375,000, and 1,000,000 states in this experiment, but it is skipped at 7,500,000 and 20,000,000 states.

Table II
REPEATED SYNTHETIC KNAPSACK RESULTS (100 RUNS PER SIZE)

N	Algorithm	Mean Value	Mean Time (ms)	Mean Gap (%)	Timeout Rate
100	DP exact	35491.8	2.991	0.00	0.00
100	Greedy density	35439.0	0.022	0.15	0.00
100	Greedy value	31962.1	0.014	9.95	0.00
100	Random feasible	20124.2	0.026	43.35	0.00
100	Simulated annealing	35439.8	0.781	0.15	0.00
1000	DP exact	111281.2	56.644	0.00	0.00
1000	Greedy density	111252.4	0.209	0.03	0.00
1000	Greedy value	40785.1	0.147	63.30	0.00
1000	Random feasible	20897.2	0.173	81.20	0.00
1000	Simulated annealing	111252.4	3.784	0.03	0.00
10000	DP exact	–	–	–	1.00
10000	Greedy density	342178.6	2.001	–	0.00
10000	Greedy value	43537.4	1.582	–	0.00
10000	Random feasible	21552.6	1.701	–	0.00
10000	Simulated annealing	342178.6	10.320	–	0.00

This directly demonstrates the pseudo-polynomial limitation of dynamic programming.

Table III
DP SCALABILITY AS nC GROWS

N	Capacity	nC States	Timeout Rate
100	500	50000	0.00
500	750	375000	0.00
1000	1000	1000000	0.00
5000	1500	7500000	1.00
10000	2000	20000000	1.00

Table IV
CHUNKING ABLATION WITH THE SAME KNAPSACK SOLVER

Chunker	Budget	Chunks	Avg. Tok.	Term Recall	Complete	Avg. Used	Util.
semantic-threshold	1%	1332	124.610	0.798	0.583	1658.667	1.000
semantic-threshold	3%	1332	124.610	0.854	0.625	4978.833	1.000
semantic-threshold	5%	1332	124.610	0.894	0.750	8278.625	0.998
structure-aware	1%	639	259.800	0.979	0.958	1653.833	0.997
structure-aware	3%	639	259.800	1.000	1.000	4976.417	0.999
structure-aware	5%	639	259.800	1.000	1.000	8296.583	1.000

Table IV adds the missing chunking ablation. Because the solver and scoring profile are fixed, differences come from the item set created before optimization. On this PDF sample, structure-aware semantic chunking dominates standalone semantic-threshold chunking at every tested budget. At the tight 1% budget, structure-aware reaches 0.979 evidence-term recall, while standalone semantic-threshold reaches 0.798. By 3%, the structure-aware variant already recovers all evidence terms, whereas standalone semantic-threshold remains below that level even at 5%. In this dataset, using semantic drift inside structural constraints is better than letting adjacent embedding changes drive chunk boundaries alone.

Table V isolates the value function. With the same structure-aware chunks and the same knapsack solver, score design matters much more than the choice between exact and heuristic selection. Cosine alone reaches only 0.410 evidence-term recall and 0.083 complete-evidence rate at the tight 1% budget. Richer profiles improve sharply: the plain hybrid score reaches 0.588 recall, knapsack-aware reaches 0.826, and evidence-hybrid reaches 0.979 with a 0.958 complete-evidence rate. By 3%, several richer profiles already recover all evidence terms.

Table VI evaluates the central selector question on 861 QASPER validation questions from 276 papers. The scoring

TokenPack result dashboard

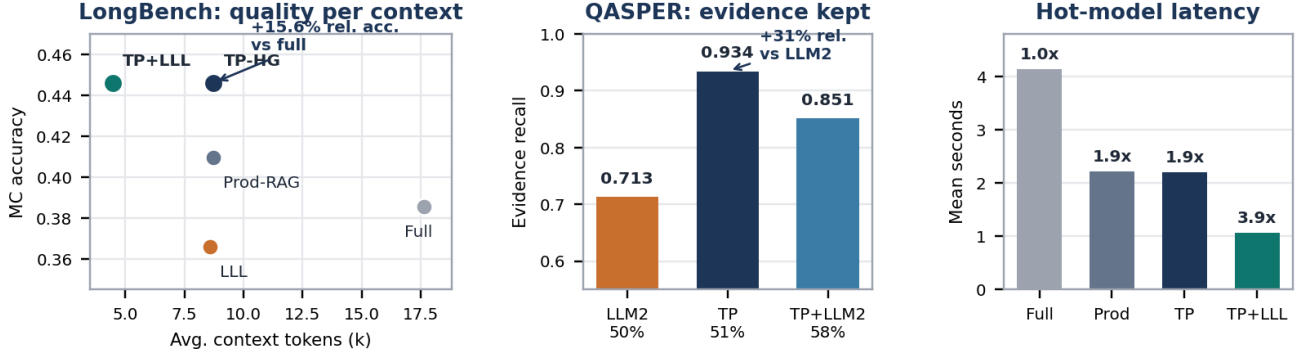


Figure 1. Headline TokenPack results. Left: LongBench v2 operating points show accuracy versus average context size. Middle: QASPER evidence recall at comparable or stronger token savings. Right: hot-model latency for the same 83-case LongBench v2 pilot.

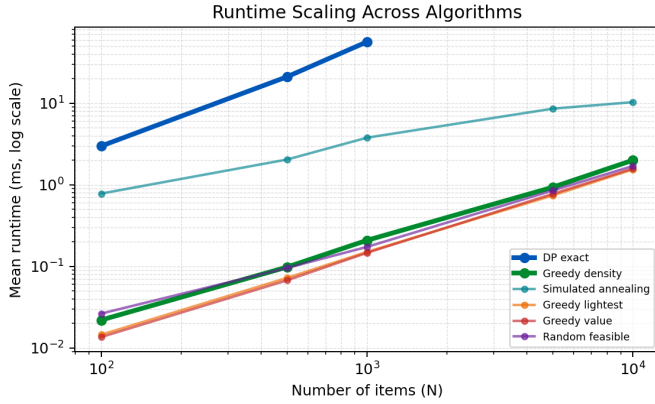


Figure 2. Runtime scaling of exact and heuristic algorithms across problem sizes.

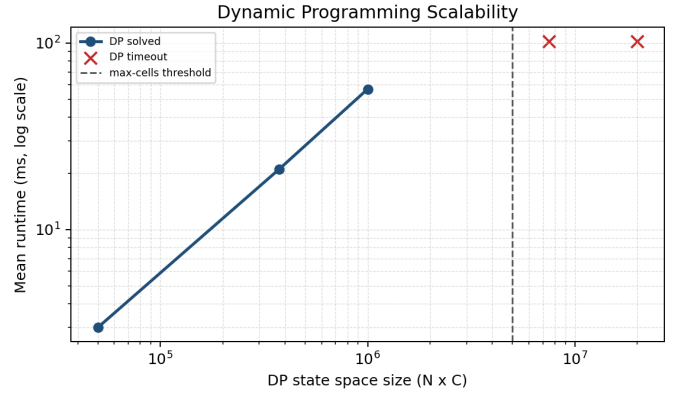


Figure 4. Dynamic programming runtime and timeout behavior as the nC state space grows.

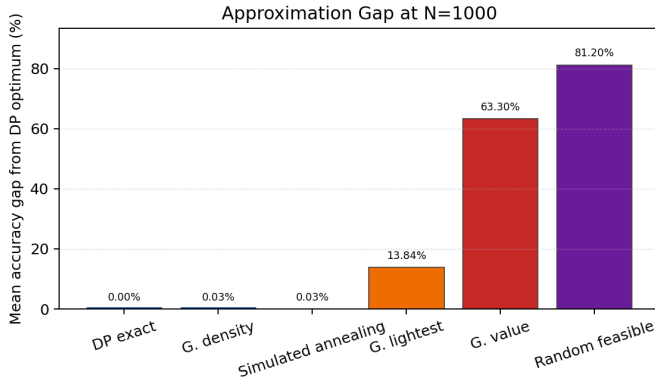


Figure 3. Mean approximation gap from the DP optimum for $N = 1000$ over 100 paired runs.

function and chunker are fixed to the strongest tested configuration, so differences come from the selection rule rather than from chunk boundaries. The results are important precisely because they are not a clean win for knapsack: budget-top-k gives the highest evidence recall at every tested budget, while knapsack and knapsack-redundancy are close but slightly lower.

Table V
VALUE FUNCTION ABLATION WITH FIXED CHUNKING AND KNAPSACK SOLVER

Scoring	Budget	Chunks	Avg. Tok.	Term Recall	Complete	Avg. Used	Redund.
cosine	1%	639	259,800	0.410	0.083	1652.750	0.021
cosine	3%	639	259,800	0.811	0.625	4975.583	0.020
cosine	5%	639	259,800	0.923	0.833	8296.958	0.021
hybrid	1%	639	259,800	0.588	0.333	1654.792	0.022
hybrid	3%	639	259,800	1.000	1.000	4975.500	0.020
hybrid	5%	639	259,800	1.000	1.000	8295.833	0.021
evidence-hybrid	1%	639	259,800	0.979	0.958	1653.833	0.025
evidence-hybrid	3%	639	259,800	1.000	1.000	4976.417	0.026
evidence-hybrid	5%	639	259,800	1.000	1.000	8296.583	0.028
knapsack-aware	1%	639	259,800	0.826	0.708	1654.833	0.023
knapsack-aware	3%	639	259,800	1.000	1.000	4975.458	0.021
knapsack-aware	5%	639	259,800	1.000	1.000	8296.125	0.023

At the same time, the knapsack variants remain competitive and are usually above or near density-greedy. This result changes the interpretation of the application case study. The classical knapsack solver is valuable for algorithm analysis and for budget-feasible selection, but it should not be presented as the universally best LLM retrieval rule under the current scoring function. Exact or near-exact budgeted optimization is only as good as the utility model it receives.

Table VII evaluates the practical cost-quality trade-off more directly. The 100% budget row is treated as the high-

Table VI
QASPER SELECTOR EVALUATION WITH FIXED SCORING. PROD-RAG
USES RAW DENSE SIMILARITY; BUDGET-TOP-K IS TOKENPACK
HYBRID-GREEDY.

Selector	Budget	Avg Tok.	Evidence Recall	Complete	Answer Overlap F1	Value	Lat.
budget-top-k	20%	982.549	0.771	0.525	0.029	2.429	0.000
greedy-density	20%	982.549	0.731	0.443	0.029	2.625	0.000
knapsack	20%	982.549	0.750	0.476	0.028	2.670	0.002
knapsack-red.	20%	982.549	0.748	0.473	0.028	2.337	0.011
Prod-RAG	20%	982.549	0.758	0.515	0.029	2.336	0.000
budget-top-k	30%	1473.954	0.853	0.684	0.021	3.380	0.000
greedy-density	30%	1473.954	0.827	0.612	0.021	3.581	0.000
knapsack	30%	1473.954	0.834	0.626	0.021	3.628	0.004
knapsack-red.	30%	1473.954	0.835	0.628	0.021	3.132	0.012
Prod-RAG	30%	1473.954	0.842	0.659	0.021	3.264	0.000
budget-top-k	40%	1965.483	0.903	0.790	0.017	4.230	0.000
greedy-density	40%	1965.483	0.890	0.747	0.017	4.413	0.000
knapsack	40%	1965.483	0.895	0.751	0.017	4.459	0.005
knapsack-red.	40%	1965.483	0.892	0.746	0.017	3.817	0.013
Prod-RAG	40%	1965.483	0.898	0.771	0.017	4.094	0.000
budget-top-k	50%	2457.100	0.936	0.865	0.014	4.991	0.000
greedy-density	50%	2457.100	0.931	0.850	0.014	5.154	0.000
knapsack	50%	2457.100	0.936	0.856	0.014	5.198	0.007
knapsack-red.	50%	2457.100	0.936	0.857	0.014	4.423	0.015
Prod-RAG	50%	2457.100	0.931	0.852	0.014	4.855	0.000

Table VII
QASPER COST-QUALITY PROXY UNDER REDUCED CONTEXT BUDGETS.
TP-HG DENOTES TOKENPACK HYBRID-GREEDY.

Selector	Budget	Used Tok.	Saving	Ans. vs 100	Ev. vs 100	Preserved
TP-HG	50%	2420	51.0%	0.942	0.930	0.674
knapsack	50%	2434	50.7%	0.944	0.931	0.677
knapsack-red.	50%	2434	50.7%	0.944	0.931	0.684
Prod-RAG	50%	2417	51.1%	0.944	0.926	0.675
TP-HG	70%	3396	31.2%	0.981	0.978	0.786
knapsack	70%	3410	30.9%	0.981	0.977	0.785
knapsack-red.	70%	3409	30.9%	0.979	0.975	0.784
Prod-RAG	70%	3388	31.4%	0.974	0.969	0.765
TP-HG	80%	3881	21.3%	0.991	0.989	0.815
knapsack	80%	3892	21.1%	0.988	0.987	0.808
knapsack-red.	80%	3891	21.2%	0.989	0.987	0.812
Prod-RAG	80%	3872	21.6%	0.987	0.983	0.804
TP-HG	100%	4812	2.3%	1.000	1.000	0.850
knapsack	100%	4811	2.3%	1.000	1.000	0.850
knapsack-red.	100%	4808	2.4%	1.000	1.000	0.850
Prod-RAG	100%	4791	2.8%	1.000	1.000	0.847

Table VIII
QASPER COMPRESSION COMPARISON AT APPROXIMATELY MATCHED
50% TOKEN SAVING. LLM2 DENOTES LLMLINGUA-2.

Method	Saving	Evidence Recall	Complete	Answer Overlap F1	Sec.
Only LLM2-0.50	50.6%	0.713	0.120	0.010	7.04
Only TokenPack	51.1%	0.934	0.870	0.013	0.00
TokenPack + LLM2-0.50	75.9%	0.632	0.035	0.018	3.59

budget RAG reference. Under the stronger structure-aware and evidence-hybrid configuration, hybrid-greedy, knapsack, and knapsack-redundancy all preserve most of the answerability proxy at moderate budgets, but hybrid-greedy gives the cleanest practical frontier because it matches or exceeds the heavier selectors while avoiding dynamic-programming cost. At 80% budget, hybrid-greedy retains 0.989 evidence recall relative to the high-budget reference while saving 21.3% of tokens. At 70% budget, it retains 0.978 evidence recall with 31.2% token saving. At 50% budget, it still retains 0.930 evidence recall at 51.0% token saving. The practical lesson is conservative but useful: selector optimization helps, and under the current scoring function the strongest retrieval-side rule is evidence-hybrid budget-top-k rather than the older knapsack-redundancy default.

Table VIII compares retrieval-time selection with token-level prompt compression on the same 200 QASPER validation

Table IX
COMPRESSION FRONTIER ON QASPER (ALL ROWS USE 200 QUESTIONS).
LLM2 DENOTES LLMLINGUA-2.

Method	Saving	Evidence Recall	Answer Overlap F1	Run
Only LLM2-0.50	50.6%	0.713	0.010	current-200q
Only TokenPack	51.1%	0.934	0.013	current-200q
TokenPack + LLM2-0.85	58.4%	0.851	0.014	current-200q
TokenPack + LLM2-0.75	63.5%	0.792	0.015	current-200q
TokenPack + LLM2-0.50	75.9%	0.632	0.018	current-200q

Run notes: all rows are refreshed *current-200q* runs on the same 200-question split.

questions at approximately matched 50% token savings. Only TokenPack saves 51.1% of input tokens while preserving 0.934 evidence recall. Only LLMLingua-2 reaches a nearly identical 50.6% saving, but its evidence recall drops to 0.713. The complete-evidence rate gap is even larger: 0.870 for TokenPack versus 0.120 for LLMLingua-2. The answer-token-overlap F1 values stay numerically small because they measure answer-string retention inside the selected context, not generated answer correctness. This is the clearest application-side result of the paper: when the final budget is matched, structure-aware evidence selection preserves answer-bearing spans better than token-level compression alone.

Table IX extends the comparison to stronger second-stage compression. All rows are refreshed on the same 200-question QASPER validation split with structure-aware chunks, evidence-hybrid scoring, hybrid-greedy selection, and LLMLingua-2 as the second-stage compressor. TokenPack followed by LLMLingua-2 at rate 0.85 preserves 0.851 evidence recall while reaching 58.4% total token saving, which is better than Only LLMLingua-2 at 50.6% saving and 0.713 evidence recall. More aggressive second-stage compression continues to increase savings, but the frontier shows a consistent pattern: the first quality-preserving step is retrieval-time evidence selection, while prompt compression is best interpreted as an optional second-stage cost reducer with a visible recall trade-off.

Table X
EXPLORATORY 83-CASE LONGBENCH V2 MULTIPLE-CHOICE PILOT WITH
QWEN/QWEN2.5-14B-INSTRUCT. TP-HG DENOTES TOKENPACK
HYBRID-GREEDY; LLL DENOTES LONGLLMLINGUA CONTEXT-LEVEL
FILTERING; -50 DENOTES AN APPROXIMATELY 50% SOURCE-TOKEN
SELECTION BUDGET. REL. VS FULL IS DESCRIPTIVE AND IS NOT A
STATISTICALLY POWERED SIGNIFICANCE CLAIM.

Method	Runs	Correct	Acc.	Rel. vs Full	Avg. Ctx.	Saving
Full context	83	32/83	0.386	–	17646	0.000
Prod-RAG-50	83	34/83	0.410	+6.2%	8720	0.506
TP-HG-50	83	37/83	0.446	+15.6%	8731	0.506
LLL-0.50	83	30/82	0.366	-5.1%	8570	0.511
TP-HG-50 + LLL-0.50	83	37/83	0.446	+15.6%	4468	0.746

Table X adds an exploratory end-to-end long-context generation pilot. Unlike the QASPER compression tables, this experiment measures final multiple-choice correctness rather than evidence-retention proxies, but it is still a small descriptive study rather than a statistically powered benchmark. On 83 eligible LongBench v2 cases, full-context prompting answers 32 cases correctly (0.386), production-RAG answers 34 correctly (0.410) at roughly half the input size, and TokenPack hybrid-greedy answers 37 correctly (0.446) at the same budget

scale. This corresponds to a +6.0 percentage-point gain, or +15.6% relative accuracy, over full-context prompting, and a +3.6 percentage-point gain, or +8.8% relative accuracy, over production-RAG-50. These differences are small relative to the roughly ± 0.11 binomial uncertainty of an 83-case pilot, so the table should be read as hypothesis-generating evidence rather than as a statistically significant method ranking. The clearer systems signal is the cascade: TokenPack followed by LongLLMLingua reaches 74.6% saving while retaining the same 37/83 correctness as TokenPack alone.

Table XI

LATENCY COMPANION TO THE EXPLORATORY 83-CASE LONGBENCH V2 PILOT WITH QWEN/QWEN2.5-14B-INSTRUCT. TP-HG DENOTES TOKENPACK HYBRID-GREEDY; LLL DENOTES LONGLLMLINGUA CONTEXT-LEVEL FILTERING; -50 DENOTES AN APPROXIMATELY 50% SOURCE-TOKEN SELECTION BUDGET. LATENCY IS MEASURED WITH BATCH SIZE ONE AND EXCLUDES MODAL COLD START/MODEL LOADING.

Method	Runs	Saving	Prep s	LLM s	Total s	Speedup
Full context	83	0.000	0.000	4.140	4.140	1.00×
Prod-RAG-50	83	0.506	0.000	2.223	2.223	1.86×
TP-HG-50	83	0.506	0.000	2.196	2.196	1.89×
LLL-0.50	83	0.511	0.523	1.914	2.437	1.70×
TP-HG-50 + LLL-0.50	83	0.746	0.261	0.800	1.060	3.90×

Table XI reruns the same 83-case LongBench v2 setting with batch size one to measure hot-model wall-clock latency per request. The measurement excludes Modal cold start and model loading, but includes the TokenPack selection and LongLLMLingua preprocessing time that occurs before the answer request. TokenPack alone reduces mean total latency from 4.140s to 2.196s, a 1.89 $\times$ speedup. The TokenPack-then-LongLLMLingua cascade reduces the final context to 4,468 tokens on average and reaches 1.060s mean total latency, a 3.90 $\times$ speedup over full-context prompting.

Table XII and Figure 5 report the larger-context diagnostic. The 32k–58k window gives the most useful extra signal: TP-HG-50 reaches 0.500 accuracy at 50.2% saving and 2.15 $\times$ speedup, compared with 0.433 for production-RAG-50 and 0.400 for full context. The 64k–112k window confirms that the same half-budget operating point produces roughly 50% token savings and more than 2 $\times$ latency reduction, but the sample has only 8 cases and should not be used to rank selectors. Pairwise diagnostics from the current 83-case run provide an additional sanity check: against production-RAG-50, TP-HG-50 records 8 wins, 70 ties, and 5 losses, while TP-HG-50 plus LongLLMLingua records 10 wins, 66 ties, and 7 losses.

Table XIII and Figure 6 show the economic implication of the LongBench v2 context-reduction ratios. TokenPack alone reduces the scaled input from 1,000,000 to roughly 494,000 paid tokens, while the TokenPack-then-LongLLMLingua cascade reduces it to roughly 254,000 paid tokens, corresponding to the 74.6% average saving reported in Table X. The exact dollar value changes with provider pricing, but the token reduction ratio is determined by the selection and compression strategy.

Table XII

LONGBENCH V2 LARGER-CONTEXT SCALING DIAGNOSTIC WITH QWEN/QWEN2.5-14B-INSTRUCT. HG DENOTES EVIDENCE-HYBRID GREEDY FILL; DP DENOTES THE DIAGNOSTIC DYNAMIC-PROGRAMMING SELECTOR AFTER THE STATED VALUE FUNCTION. THESE LARGER-WINDOW RUNS ARE FEASIBILITY DIAGNOSTICS ONLY, NOT PRIMARY ACCURACY EVIDENCE OR A STABLE SELECTOR RANKING.

Source Window	Method	Cases	Acc.	Correct	Avg. Ctx.	Saving	Total s	Speedup
32k–58k	Full context	30	0.400	12/30	43704	0.000	13.580	1.00×
32k–58k	Prod-RAG-50	30	0.433	13/30	21770	0.502	6.440	2.11×
32k–58k	Dense DP-50	30	0.300	9/30	21813	0.501	6.883	1.97×
32k–58k	TP-HG-50	30	0.500	15/30	21767	0.502	6.313	2.15×
32k–58k	Hybrid DP-50	30	0.367	11/30	21803	0.501	6.718	2.02×
64k–112k	Full context	8	0.375	3/8	78844	0.000	9.317	1.00×
64k–112k	Prod-RAG-50	8	0.500	4/8	39300	0.501	4.049	2.30×
64k–112k	Dense DP-50	8	0.375	3/8	39401	0.500	4.530	2.06×
64k–112k	TP-HG-50	8	0.375	3/8	39290	0.501	3.991	2.33×
64k–112k	Hybrid DP-50	8	0.500	4/8	39340	0.501	4.666	2.00×

Table XIII

ESTIMATED INPUT COST SAVINGS FROM LONGBENCH V2 CONTEXT REDUCTION. TP-HG DENOTES TOKENPACK HYBRID-GREEDY.

Method	Paid Tokens	Cost (\$)	Saving vs Full Doc
Full context	1000000	2.500	0.0%
Prod-RAG-50	493674	1.234	50.6%
TP-HG-50	494369	1.236	50.6%
LLL-0.50	488787	1.222	51.1%
TP-HG-50 + LLL-0.50	254293	0.636	74.6%

LongBench larger-context diagnostic

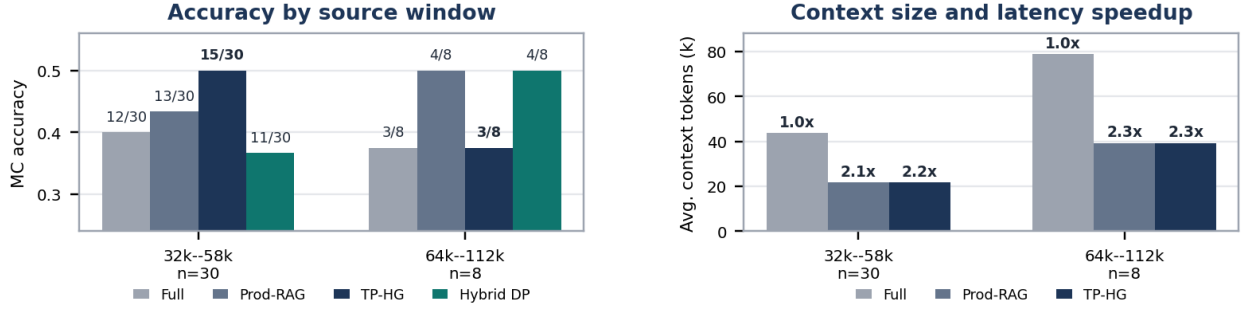


Figure 5. Visual companion to the larger-context LongBench v2 diagnostic in Table XII. The 32k–58k window has 30 cases; the 64k–112k window has only 8 cases and is included as a scaling feasibility check rather than a stable selector ranking.

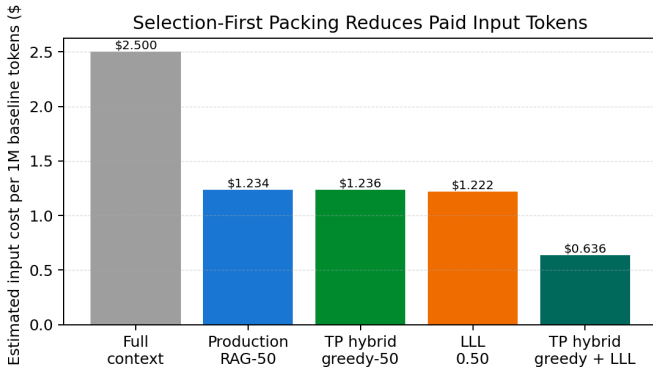


Figure 6. Estimated input-token cost under a 1M baseline-token scenario.

VIII. DISCUSSION

The experiments support seven observations. First, exact DP is the most reliable method when the instance size is manageable because it gives the optimum. Second, the $O(nC)$ cost becomes a real limitation as the state space grows: in the repeated benchmark, DP is feasible for the smaller settings but is skipped at the larger state counts. Third, density-aware greedy and simulated annealing are strong practical approximations on the tested solvable sizes because they retain most of the objective value at much lower cost. Fourth, value-only greedy can fail badly, even though it sounds intuitive, because it ignores weight. Fifth, the LLM case study shows that budgeted selection and compression should not be conflated: selecting evidence-bearing chunks before prompting is a different operation from pruning prompt tokens after context construction. Sixth, the LongBench v2 pilot offers hypothesis-generating evidence that this selection-first framing can transfer from evidence-retention proxies to downstream answering: TokenPack hybrid-greedy improves pilot accuracy by 15.6% relative to full-context prompting while saving about half the source tokens, and the selection-then-compression cascade keeps the same 37/83 correctness while pushing the average saving to 74.6%. Seventh, the larger-context diagnostics show that the same source-relative half-budget regime is operationally feasible beyond the 32k

main pilot, although the 64k–112k sample is too small for a stable method ranking.

For LLM context selection, these observations imply that TokenPack should not run DP over every chunk in a huge corpus. Instead, the retrieval system should first create a candidate pool using embeddings, then apply DP or a strong approximation inside that smaller pool. The QASPER selector results also clarify what TokenPack is and is not proving at this stage. The current paper does not show that knapsack is the universally best retrieval rule under evidence recall. Rather, it shows that budgeted selection is the right abstraction, that solver quality and score quality are separable, and that a stronger score model is now the main limiting factor. In fact, the most robust current retrieval-side rule is the simpler evidence-hybrid budget-top-k selector. Therefore, the appropriate claim is not “knapsack beats retrieval,” but “the 0/1 knapsack framework enables exact-vs-heuristic algorithm analysis, and the LLM setting is a realistic budgeted-selection case study.” The compression and LongBench comparisons refine the claim further: TokenPack does not make the language model itself smarter, but it selects an evidence substrate and can improve the operating point of a downstream compressor. Prompt compression can still be valuable after selection, especially when a system must push to higher savings, yet the QASPER frontier and the LongBench cascade both suggest that selection-first pipelines are a better framing than treating compression as the only budget-control step.

IX. LIMITATIONS AND FUTURE WORK

The current results are strongest on synthetic knapsack instances, where the optimum and approximation gap are clearly defined. The LLM experiments should be treated as an application case study rather than as proof of universal retrieval dominance. QASPER measures retention proxies rather than final human-judged answer quality, and the LongBench accuracy differences are descriptive rather than statistically significant at this sample size. The larger-context LongBench rows are even more diagnostic: the 32k–58k window has 30 cases and the 64k–112k window has only 8 cases. A

further limitation is the production scoring function itself: although the evidence-hybrid formula exposes its component weights, this version does not derive those weights from a held-out calibration set, a learned ranker, or a sensitivity study. Future work should tune or learn these coefficients and report how stable the selector rankings are under reasonable weight changes. The compression comparison focuses on one family of methods, uses LongLLMLingua context-level filtering rather than the full token-level compressor, and reports realized savings but not full per-document rate variance. BudgetMem is discussed as related work, but no direct numerical BudgetMem baseline is included because the study did not have an official implementation or exact reproduction artifact; the artifact includes only a BudgetMem-style feature proxy, which trails evidence-hybrid on QASPER. A larger study should add full LongLLMLingua token-level compression, a true BudgetMem reproduction if code becomes available, KV-compression baselines, more generator models, human review of disagreement cases, and learned or tuned scoring and chunk-size policies.

X. CONCLUSION

This paper evaluates the 0/1 Knapsack Problem through exact and heuristic algorithms and uses LLM context packing as a realistic application of budgeted selection. The algorithm-analysis result is clear: exact DP is optimal but scales with the pseudo-polynomial $O(nC)$ state space, while density-aware greedy and simulated annealing are much faster and stay close to the optimum on the tested solvable instances. The application-side result is intentionally nuanced: hybrid-greedy remains the strongest QASPER evidence-recall baseline under the current scoring function, so the paper does not claim universal knapsack dominance for LLM retrieval. Instead, the LLM experiments show why budgeted selection is useful and why retrieval-time selection and prompt compression are not interchangeable. TokenPack preserves substantially more QASPER evidence than LLMLingua-2 at matched 50% token savings. The LongBench v2 pilot is best read as descriptive case-study evidence: TokenPack hybrid-greedy improves accuracy by 15.6% relative to full-context prompting while using about half the context, and the TokenPack-then-LongLLMLingua cascade preserves the same 37/83 correctness while increasing context saving to 74.6%. Future gains will depend on utility models better aligned with downstream answer quality and on larger generation benchmarks.

REFERENCES

- [1] R. M. Karp, "Reducibility among combinatorial problems," in *Complexity of Computer Computations*. Springer, 1972, pp. 85–103.
- [2] S. Martello and P. Toth, *Knapsack Problems: Algorithms and Computer Implementations*. John Wiley & Sons, 1990.
- [3] V. Cacchiani, M. Iori, A. Locatelli, and S. Martello, "Knapsack problems: An overview of recent advances. part ii: Multiple, multidimensional, and quadratic knapsack problems," *Computers & Operations Research*, 2022.
- [4] T. Lust and J. Teghem, "The multiobjective multidimensional knapsack problem: A survey and a new approach," *International Transactions in Operational Research*, 2010.
- [5] T. Szkaliczki, "Solution methods for the multiple-choice knapsack problem and their applications," *Mathematics*, 2025.
- [6] S. Verma, M. Pant, and V. Snasel, "A comprehensive review on nsga-ii for multi-objective combinatorial optimization problems," *IEEE Access*, 2021.
- [7] P. Lewis, E. Perez, A. Piktus *et al.*, "Retrieval-augmented generation for knowledge-intensive nlp tasks," in *Advances in Neural Information Processing Systems*, 2020.
- [8] C. Packer, V. Fang, S. G. Patil, K. Lin, S. Wooders, and J. Gonzalez, "Memgpt: Towards llms as operating systems," 2023.
- [9] N. Ratner, Y. Levine, Y. Belinkov, O. Ram, I. Magar, O. Abend, E. Karpas, A. Shashua, K. Leyton-Brown, and Y. Shoham, "Parallel context windows for large language models," in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, 2023.
- [10] Y. Ding, L. Zhang, C. Zhang, Y. Xu, N. Shang, J. Xu, F. Yang, and M. Yang, "Longrope: Extending llm context window beyond 2 million tokens," 2024.
- [11] D. Zhu, N. Yang, L. Wang, Y. Song, W. Wu, F. Wei, and S. Li, "Pose: Efficient context window extension of llms via positional skip-wise training," 2023.
- [12] S. Chen, S. Wong, L. Chen, and Y. Tian, "Extending context window of large language models via positional interpolation," 2023.
- [13] P. Xu, W. Ping, X. Wu, L. C. McAfee, C. Zhu, Z. Liu, S. Subramanian, E. Bakhturina, M. Shoenybi, and B. Catanzaro, "Retrieval meets long context large language models," 2023.
- [14] X. Li, Y. Cao, Y. Ma, and A. Sun, "Long context vs. rag for llms: An evaluation and revisits," 2024.
- [15] N. Paulsen, "Context is what you need: The maximum effective context window for real world limits of llms," 2025.
- [16] A. Ahmed, S. Di, F. Cappello, Z. Liu, J. Han, and A. Anwar, "Systematic evaluation of optimization techniques for long-context language models," 2025.
- [17] M. Gunther, I. Mohr, D. J. Williams, B. Wang, and H. Xiao, "Late chunking: Contextual chunk embeddings using long-context embedding models," 2024.
- [18] P. Verma, "S2 chunking: A hybrid framework for document segmentation through integrated spatial and semantic analysis," 2025.
- [19] K. Luo, Z. Liu, S. Xiao, and K. Liu, "Landmark embedding: A chunking-free embedding method for retrieval augmented long-context large language models," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024.
- [20] —, "Bge landmark embedding: A chunking-free embedding method for retrieval augmented long-context large language models," 2024.
- [21] M. A. Shaukat, M. Adnan, and C. C. N. Kuhn, "A systematic investigation of document chunking strategies and embedding sensitivity," arXiv preprint arXiv:2603.06976, 2026.
- [22] Y. Fang, T. Sun, Y. Shi, and X. Gu, "Attentionrag: Attention-guided context pruning in retrieval-augmented generation," arXiv preprint arXiv:2503.10720, 2025.
- [23] J. Tang, Y. Zhao, K. Zhu, G. Xiao, B. Kasikci, and S. Han, "Quest: Query-aware sparsity for efficient long-context llm inference," 2024.
- [24] Y. Li, Y. Huang, B. Yang, B. Venkitesh, A. F. Locatelli, H. Ye, T. Cai, P. Lewis, and D. Chen, "Snapkv: Llm knows what you are looking for before generation," 2024.
- [25] Y. Zuo, S. Wei, C. Zhang, Z. Liu, W. Lu, and D. Song, "Windowkv: Task-adaptive group-wise kv cache window selection for efficient llm inference," 2025.
- [26] C. V. K. Alla, H. N. Gaddam, and M. Kommi, "Budgetmem: Learning selective memory policies for cost-efficient long-context processing in language models," arXiv preprint arXiv:2511.04919, 2025.
- [27] H. Jiang, Q. Wu, C.-Y. Lin, Y. Yang, and L. Qiu, "Llmlingua: Compressing prompts for accelerated inference of large language models," 2023.
- [28] H. Jiang, Q. Wu, X. Luo, D. Li, C.-Y. Lin, Y. Yang, and L. Qiu, "Longllmlingua: Accelerating and enhancing llms in long context scenarios via prompt compression," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024.
- [29] Z. Pan, Q. Wu, H. Jiang, M. Xia, X. Luo, J. Zhang, Q. Lin, V. Rühle, Y. Yang, C.-Y. Lin, H. V. Zhao, L. Qiu, and D. Zhang, "Llmlingua-2: Data distillation for efficient and faithful task-agnostic prompt compression," 2024.
- [30] T. Hirao, Y. Yoshida, M. Nishino, N. Yasuda, and M. Nagata, "Single-document summarization as a tree knapsack problem," in *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing*, 2013.

- [31] J. Pilault, R. Li, S. Subramanian, and C. Pal, “Factorizing content and budget decisions in abstractive summarization of long documents,” 2022.
- [32] A. Duchnowski, E. Pavlick, and A. Koller, “A knapsack by any other name: Presentation impacts llm performance on np-hard problems,” in *Findings of the Association for Computational Linguistics: EMNLP*, 2025.
- [33] Z. Li, C. Chen, T. Yang, T. Ding, R. Sun, G. Zhang, W. Huang, and Z. Luo, “Knapsack rl: Unlocking exploration of llms via optimizing budget allocation,” 2025.
- [34] S.-H. Cha, G. Kim, D. Han, T. Yang, and I. Han, “Knapspec: Self-speculative decoding via adaptive layer selection as a knapsack problem,” 2026.
- [35] N. Reimers and I. Gurevych, “Sentence-bert: Sentence embeddings using siamese bert-networks,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*. Association for Computational Linguistics, 2019, pp. 3982–3992.
- [36] Sentence Transformers, “all-minilm-l6-v2,” <https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2>, 2024, accessed: 2026-05-13.
- [37] P. Dasigi, K. Lo, I. Beltagy, A. Cohan, N. A. Smith, and M. Gardner, “A dataset of information-seeking questions and answers anchored in research papers,” in *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Association for Computational Linguistics, 2021, pp. 4599–4610.
- [38] Y. Bai, S. Tu, J. Zhang, H. Peng, X. Wang, X. Lv, S. Cao, J. Xu, L. Hou, Y. Dong, J. Tang, and J. Li, “Longbench v2: Towards deeper understanding and reasoning on realistic long-context multitasks,” *arXiv preprint arXiv:2412.15204*, 2024.
- [39] Qwen Team, “Qwen2.5: A party of foundation models,” <https://qwenlm.github.io/blog/qwen2.5/>, 2024.
- [40] OpenAI, “Api pricing,” <https://openai.com/api/pricing/>, 2026, accessed: 2026-05-05.